

Cross-Residual Graph Neural Networks for Biomolecular Graph Representation Learning

Jingchao Wang¹, Wei Huang¹, Xuelin Hu², and Xiaoqin Fu²

¹School of Computer Science, Zhongyuan University of Technology, Zhengzhou 450000, China

²School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou, Guangxi, China

Corresponding author:
Xuelin Hu¹

Email address: huxuelinai@gmail.com

ABSTRACT

Graph neural networks are increasingly used in biomolecular and plant-related graph learning, yet deep message passing can blur local structural cues while simple skip connections do not test richer information reuse. This limitation is especially relevant for biological graph classification, where predictive signals often depend on both local motifs and graph-level context. We therefore study Cross-Residual Graph Neural Networks (CR-GNN), a compact model family that compares five information-flow patterns under a shared backbone: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Experiments on a protein-oriented benchmark centered on PROTEINS, DD, and ENZYMES, with MUTAG, AIDS, and Mutagenicity as supplementary robustness checks, show a consistent picture. Residual reuse is the strongest default strategy, while cross-residual routing usually improves over plain stacking and delivers dataset-dependent gains on selected graphs. These results position CR-GNN as a practical biomolecular graph representation framework and clarify when cross-path interaction is useful in biologically motivated graph learning.

INTRODUCTION

Graph neural networks (GNNs) are now a core tool for structure-aware learning on molecules, proteins, and biological interaction systems [17, 12, 15]. In the biological domain, graph representations are valuable because they preserve both local neighborhood structure and larger relational context, which is critical for tasks such as protein function prediction, protein–protein interface analysis, enzyme classification, and biomolecular interaction modeling. Recent studies have already shown concrete biological impact: graph convolutional models have been used for structure-based protein function prediction [13], large-scale multispecies protein annotation [37], and protein–protein interface learning [29]. The same trend is increasingly visible in plant computational biology, where GNN-style modeling is being explored for plant protein identification, functional annotation, and biological network analysis [31, 5, 27].

Despite this progress, deep biological graph learning remains difficult. Repeated message passing can over-smooth node states, weaken discriminative local motifs, and reduce intermediate diversity, especially when the graph label depends on multiple coordinated substructures rather than on one dominant pattern. This issue is especially relevant in protein-related graphs and plant-oriented graph inference, where useful evidence may be distributed across local regions, cross-scale interactions, and graph-level summaries. A model that only stacks layers sequentially may underuse such complementary information, while a simple residual skip only re-injects the same branch history and does not test whether cross-branch exchange or graph-level feedback can preserve more useful context.

Modern GNN research has proposed several mechanisms to improve deep training, including residual connections [16, 2], normalization [38, 3], edge dropout [30], and multi-scale readout designs such as Jumping Knowledge [35]. These methods improve optimization or preserve multi-depth features, but

most still operate inside a single branch. In parallel, graph-level pooling and hierarchical modules improve global summarization, yet they do not directly test whether exchanging intermediate states across branches or reusing graph summaries across block boundaries yields a better trade-off for biomolecular graph inference.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a unified family of graph classification models that isolates three kinds of information reuse: intra-branch residual accumulation, cross-branch node-state exchange, and graph-level summary propagation across sequential blocks. Rather than presenting one oversized architecture, we formulate a controlled model family that includes plain, residual, and cross-residual variants under the same message-passing backbone. This makes it possible to ask a more useful question than whether one model wins everywhere: when does cross-residual interaction help beyond a simpler residual baseline on biomolecular graphs, and when does residual reuse remain the better default for protein-oriented graph inference?

This framing also clarifies the scope of the study. The present work does not claim to solve full integrative omics modeling or novel biological discovery directly. Instead, it focuses on graph representation learning for a protein-oriented benchmark package spanning PROTEINS, DD, and ENZYMES, and studies how information reuse can preserve biologically meaningful structure more effectively than plain stacking. In this sense, the paper should be read as a biomolecular graph learning study with a technically motivated extension path toward plant-related graph inference settings in which graph topology can later be combined with richer functional or omics annotations.

Related work

The motivation of CR-GNN is not only architectural but also domain-specific: biological graph learning often requires preserving local structural cues together with broader graph context. We therefore review the literature from six connected perspectives, starting with biomolecular and plant applications before moving to graph classification, message passing operators, deep-training challenges, hybrid architectures, and cross-layer aggregation.

GNNs in biomolecular and plant graph learning

Biological graph learning has moved well beyond generic graph benchmarks. In biomolecular settings, GNNs have already been used for structure-based protein function prediction [13], large-scale protein function prediction across species [37], and protein-protein interface pattern discovery [29]. These studies show that graph models are practically relevant when the target depends on coordinated structural signals rather than on isolated features. The same pattern is now emerging in plant computational biology. Recent work applies graph-based modeling to plant vacuole protein identification [31], plant biological-network analysis [5], and plant functional annotation from knowledge graphs [27]. This literature does not validate our exact cross-residual design, but it makes the application setting concrete: in both biomolecular and plant tasks, the model often needs to preserve multi-scale biological information rather than only deepen a single message-passing chain.

Benchmark choice matters in this context. TU datasets such as PROTEINS, DD, and ENZYMES remain widely used because they are easy to reproduce and still expose meaningful differences in graph scale, sparsity, and classification difficulty [26]. For the present work, these datasets provide a stable protein-oriented evaluation core while keeping the narrative aligned with real biological graph-learning use cases.

Graph classification and message passing

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [14] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks framework [12] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean or sum pooling [8]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [36] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [19] employs self-attention to identify salient nodes, and LocalPool [4] enables sparse hierarchical classification [25, 24].

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks [17], GraphSAGE [15], GIN [34], GAT [33], and graph Transformers [9] provide complementary design choices, but they are usually studied inside homogeneous architectures rather than under a unified comparison of information-flow mechanisms.

Deep GNN training and cross-layer reuse

Training deep GNNs faces fundamental challenges due to representation degradation. Over-smoothing causes node representations to converge toward indistinguishable values as depth increases [20, 6, 22], while over-squashing limits the propagation of long-range dependencies through graph bottlenecks [32, 1]. Residual connections [16, 2], normalization [38, 3, 23], and regularization such as DropEdge [30] alleviate some of these issues, and recent work has shown that very deep GNNs can be trained under suitable designs [21, 28].

Cross-layer connections also aim to capture multi-scale information. Jumping Knowledge [35], Graph U-Net [11], APPNP [18], and dense graph designs [7] all improve feature reuse, but most approaches operate within a single branch. Our work instead distinguishes between node-level residual reuse, node-level cross exchange, graph-level residual propagation, and graph-level cross-summary exchange within one shared graph-classification framework.

Contributions

The contributions of this work are summarized as follows:

- We formalize CR-GNN as a compact architecture family that compares five biologically relevant graph classification models under one shared backbone: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN.
- We design a biologically focused evaluation protocol built around a protein-oriented benchmark core (PROTEINS, DD, ENZYMES) plus supplementary robustness datasets, and we use a controlled stratified 5-fold setting to make the architectural comparison more rigorous.
- We show that cross-residual design is useful but not universally dominant: residual reuse remains the strongest default, while cross-residual routing consistently improves over plain stacking and yields selective gains that clarify when node-level or graph-level information reuse is most helpful.

MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark package, and the shared experimental protocol. The goal is to isolate where information is reused while keeping the remaining pipeline as consistent as possible.

Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$, the objective is to learn a mapping function $f_\theta : \mathcal{G} \rightarrow \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- **PlainGNN**: sequential message passing without explicit state reuse
- **NodeResGNN**: single-branch node-level residual reuse
- **NodeCrossGNN**: two-branch node-level cross exchange
- **GraphResGNN**: graph-level residual propagation across sequential blocks
- **GraphCrossGNN**: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks.

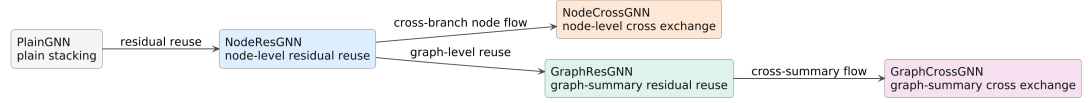


Figure 1. Main model comparison in this paper. The five architectures differ only in how intermediate node states or graph summaries are reused, which allows a controlled study of residual and cross-residual information flow.

Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_{\Phi} \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_{\mathcal{G}} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right). \quad (2)$$

The classifier produces

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}), \quad (3)$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches.

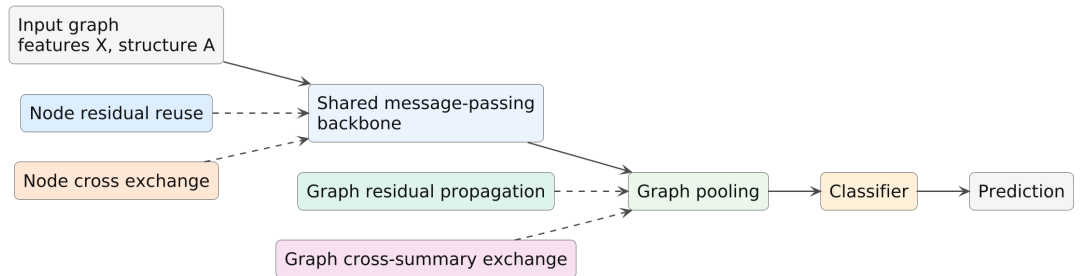


Figure 2. Schematic view of the CR-GNN pipeline. All models share the same graph-classification backbone, while node-level reuse and graph-level summary reuse define the residual and cross-residual variants.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of several standard message-passing operators,

$$\Phi \in \{\text{GCN}, \text{GAT}, \text{Graph-Transformer}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d , every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

161 **Single-branch and node-level residual variants**

162 **PlainGNN.** The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (7)$$

163 **NodeResGNN.** The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell\left(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\right)\right)\right). \quad (8)$$

164 **NodeCrossGNN.** The node-level cross-residual model maintains two parallel branches with separate
165 input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (9)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (10)$$

166 The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (11)$$

167 **Graph-level residual propagation**

168 **GraphResGNN.** This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\right), \quad t \in \{1, 2\}. \quad (12)$$

169 **GraphCrossGNN.** Four blocks are organized as two sequential pairs. Within each stage, the two
170 block outputs are swapped and reused as the next stage’s graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right), B_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right)\right), \quad (13)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}\right). \quad (14)$$

171 The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (15)$$

172 **Datasets and evaluation protocol**

173 To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into a
174 unified two-layer package.

175 The main biological benchmark consists of PROTEINS, DD, and ENZYMES [10, 26]. The supple-
176 mentary robustness package consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the
177 main biological claim centered on protein-oriented datasets while still testing structural robustness on
178 additional molecular graphs.

179 **PROTEINS** PROTEINS contains graph representations of proteins, where nodes denote secondary
180 structure elements and edges encode neighborhood relations between them. The task is binary graph
181 classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

182 **DD** DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent
183 amino acids or structure-related units linked by proximity-derived edges. The task is binary classification.
184 DD contains 1,178 graphs, 2 classes, and 89 input features.

185 **ENZYMES** ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input
186 features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable
187 to the other graph classification benchmarks in the study.

188 All active datasets are used through a unified benchmark interface [10, 26]. We keep preprocessing
189 intentionally light and do not introduce task-specific feature engineering, graph augmentation, or edge
190 redesign in the final protocol.

191 All reported results follow the same stratified two-stage evaluation design:

Table 1. Unified summary of the main biological benchmark package.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	TU	classification	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	TU	classification	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	TU	classification	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	TU	classification	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	TU	classification	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	TU	classification	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

- **Outer evaluation:** stratified 5-fold cross-validation at the graph level
- **Inner validation:** a stratified validation subset sampled from the training fold with validation ratio 0.1

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (16)$$

Across the five folds, we report the mean accuracy

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (17)$$

and the corresponding standard deviation

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (18)$$

Experimental settings

We compare the five architectures under the same GCN-style message-passing backbone, the same dataset protocol, and the same validation-based model-selection rule. All reported runs follow the same reproducible evaluation policy. The important point for the paper is that every architecture is tested under the same data protocol, the same operator backbone, and the same model-selection criterion, so the comparison isolates the effect of information-flow design rather than changes in optimization or preprocessing.

RESULTS

This section answers two questions. First, does cross-residual information flow improve graph classification beyond plain stacking? Second, when compared against simpler residual reuse, where does the cross design help and where does it remain weaker?

Full-suite benchmark results

Tables 3–5 summarize the revised benchmark package over the active biological and supplementary datasets.

Table 3. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 3 anchors the paper’s biological argument. GraphResGNN is best on PROTEINS and ENZYMES, whereas NodeResGNN is best on DD, so the core package already favors residual reuse over plain stacking. The cross-residual variants remain competitive, but on the main biological datasets they mostly narrow the gap instead of overturning the residual baseline.

Table 4. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Table 4 broadens the scope beyond the protein-oriented core. NodeCrossGNN becomes strongest on MUTAG and Mutagenicity, while GraphResGNN remains best on AIDS. This mixed pattern shows why cross-residual routing is worth keeping in the comparison, but also why the paper avoids claiming that cross routing is universally stronger than residual reuse.

Table 5. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

Table 5 turns the six benchmark outcomes into a compact cross-dataset summary. The winners change with dataset regime, the best epoch ranges from 25.4 to 131.2, and the largest parameter count does not always produce the best mean accuracy. This supports the paper’s main methodological point that information-flow design matters more than simply selecting the heaviest model.

Taken together, the three tables show a consistent hierarchy. Residual reuse is still the stronger default family in the protein-oriented core, cross-residual routing becomes selectively useful on supplementary molecular datasets, and the winner summary confirms that no single complexity trend explains the results by itself.

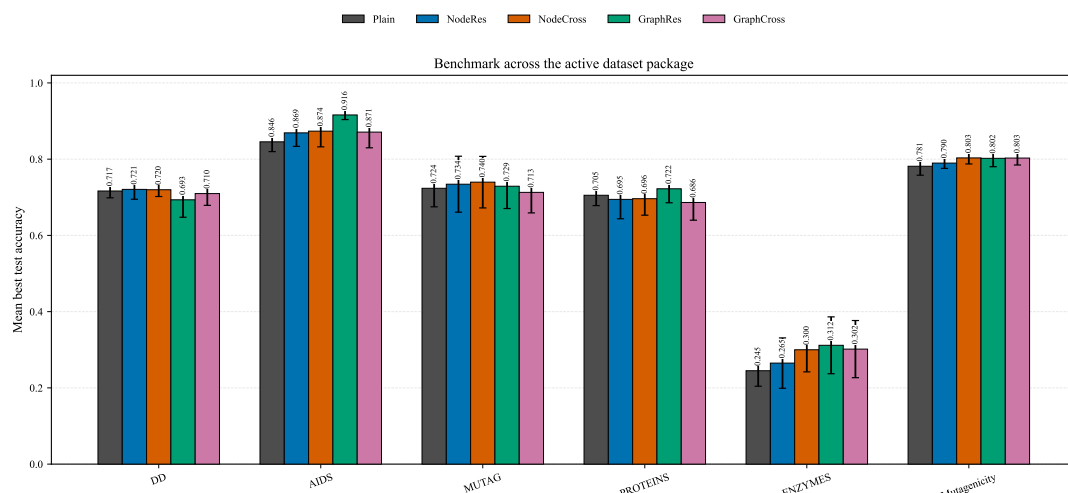


Figure 3. Benchmark over the active dataset package. Each bar reports mean best test accuracy, the numeric label above each bar gives the exact mean value, and error bars show the fold-level standard deviation.

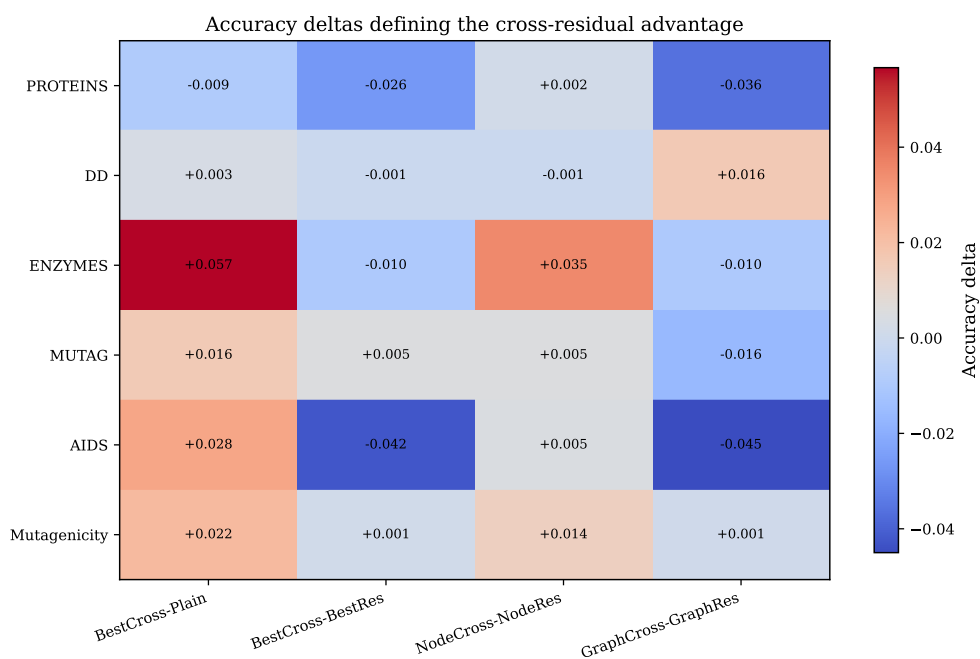


Figure 4. Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

Figure 3 and Figure 4 make the same point from complementary angles: cross-residual routing is usually better than plain stacking, but it is not consistently stronger than the best residual alternative.

231 Main findings on the protein-oriented benchmark core

232 **Residual reuse remains the strongest default family in the reported suite.** Residual variants win
 233 PROTEINS, DD, ENZYMES, and AIDS. In particular, GraphResGNN is best on PROTEINS ($0.7223 \pm$
 234 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeResGNN is best on
 235 DD (0.7207 ± 0.0259).

Cross-residual design is selectively strong, not universally dominant. Cross variants still win completed datasets such as MUTAG and Mutagenicity. More importantly, the best cross variant beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a design curiosity, but the evidence does not support the stronger claim that cross universally outperforms residual reuse.

Node-level cross is more reliable than graph-level cross. Across the completed datasets, NodeCrossGNN outperforms NodeResGNN more often than GraphCrossGNN outperforms GraphResGNN. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present study, while graph-level cross routing is more dataset-sensitive.

Plain stacking is no longer a credible final baseline. PlainGNN remains competitive on PROTEINS and DD, but it never wins a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant.

Topic-focused results for PROTEINS, DD, and ENZYMES

The topic-facing interpretation is anchored in the protein-oriented core datasets.

On **PROTEINS**, GraphResGNN is clearly strongest, indicating that repeated graph-level residual refinement is the most effective way to preserve useful global structure. The best cross model here is NodeCrossGNN at 0.6963 ± 0.0433 , which improves slightly over NodeResGNN but remains below both PlainGNN and GraphResGNN.

On **DD**, NodeResGNN and NodeCrossGNN are nearly tied, with NodeResGNN retaining a narrow lead. This indicates that node-level hidden-state reuse is important on large sparse protein graphs, but cross exchange does not produce a decisive improvement over simpler residual reuse.

On **ENZYMES**, cross variants become more attractive. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580 , both clearly above PlainGNN at 0.2450 ± 0.0407 . Even so, GraphResGNN still remains the best model.

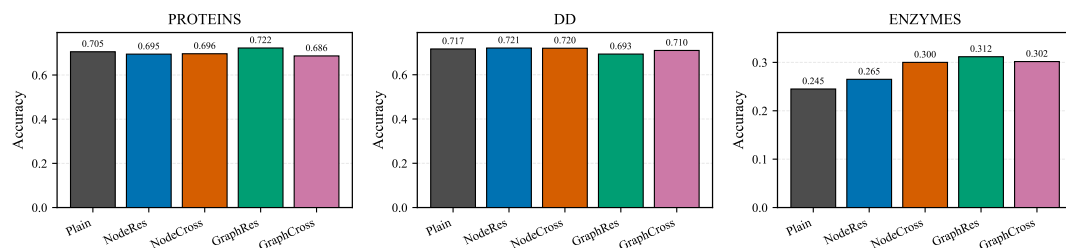


Figure 5. Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

Robustness observations

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the main conclusion. On PROTEINS and DD, moderate tuning changes affect the gap between the two cross variants, but they do not overturn the broader conclusion that residual baselines remain the stronger default reference. On ENZYMES, the validation signal is visibly noisier, and promising local settings did not reproduce the original full-suite cross results under the final 5-fold average.

DISCUSSION

The main experimental message is precise. Cross-residual routing is usually beneficial relative to plain stacking, but the gain over residual baselines is conditional and dataset-dependent. The present evidence therefore supports CR-GNN as a meaningful architectural design space rather than a universal winner.

The distinction between node-level and graph-level information reuse is also practically important. Node-level cross-residual exchange is more stable across datasets, whereas graph-level cross routing achieves stronger gains only on selected tasks. This suggests that the place where information is reused matters at least as much as the existence of additional reuse paths.

From a biological-application perspective, the strongest evidence still comes from PROTEINS, DD, and ENZYMES. These datasets support the claim that CR-GNN is useful for biomolecular graph representation learning, but they do not yet justify stronger claims about direct biological discovery or fully integrative omics prediction. The current study remains structure-centric rather than omics-integrative. This limitation also clarifies the next research step. The architecture-level findings should be tested with richer biological inputs, such as sequence-derived embeddings, ontology annotations, or expression-related signals. In parallel, the same information-flow designs can be extended toward more explicit plant-oriented graph inference tasks so that the methodological motivation in the introduction is examined under domain-specific supervision instead of benchmark transfer alone.

CONCLUSIONS

This paper reframed the cross-residual GNN study around biomolecular graph representation learning rather than generic benchmark breadth. The final result is a narrower but stronger claim: cross-residual interaction is useful, yet residual variants remain the strongest default family in the reported benchmark suite.

The main contribution is therefore not a universally dominant model, but a controlled architecture family that makes the trade-offs among plain stacking, residual reuse, and cross-residual interaction directly visible. Across the protein-oriented benchmark core, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. Cross-residual variants still provide useful gains over plain stacking, especially on ENZYMES and selected supplementary datasets, but they do not replace residual baselines as the default point of reference.

These results position CR-GNN as a practical biomolecular graph representation framework and provide a credible methodological basis for future work on richer protein and plant-related graph inference settings.

ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

A WHY RESIDUAL REUSE CAN SLOW OVER-SMOOTHING

This appendix provides a simplified derivation that explains why residual reuse can mitigate over-smoothing. The goal is not to claim a full theorem for the nonlinear models in Section , but to show that the proposed residual pathways change the effective propagation filter from a single high-order power to a lower-order polynomial mixture.

A.1 Linearized Plain Propagation

Consider the standard linearized message-passing update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\mathbf{H}^{(\ell)}, \quad (19)$$

where $\mathbf{S}$ is a normalized graph propagation operator and $\mathbf{H}^{(0)} = \mathbf{X}$. Ignoring nonlinearities and trainable weights yields

$$\mathbf{H}^{(\ell)} = \mathbf{S}^\ell \mathbf{X}. \quad (20)$$

If $\mathbf{S}$ is diagonalizable as $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$, then

$$\mathbf{H}^{(\ell)} = \mathbf{U}\mathbf{\Lambda}^\ell\mathbf{U}^{-1}\mathbf{X}. \quad (21)$$

For normalized propagation, the dominant eigenvalue is typically close to 1, while the remaining eigenvalues satisfy $|\lambda_i| < 1$. As ℓ grows, the factors λ_i^ℓ suppress non-dominant components, so node embeddings gradually collapse toward a low-dimensional smooth subspace. This is the standard over-smoothing effect: nodes become increasingly difficult to distinguish because repeated propagation attenuates the higher-frequency components that encode local differences.

315 A.2 Node-Level Residual Reuse

316 Now consider a residual reuse update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}(\mathbf{H}^{(\ell)} + \beta \mathbf{H}^{(\ell-1)}), \quad (22)$$

317 where $\beta \geq 0$ controls the strength of the reused hidden state. In this study, we set $\beta = 1$.

318 Define $\mathbf{H}^{(\ell)} = P_\ell(\mathbf{S})\mathbf{X}$ with $P_0(\mathbf{S}) = \mathbf{I}$ and $P_1(\mathbf{S}) = \mathbf{S}$. Then Eq. (22) implies the polynomial recurrence

$$P_{\ell+1}(\mathbf{S}) = \mathbf{S}(P_\ell(\mathbf{S}) + \beta P_{\ell-1}(\mathbf{S})). \quad (23)$$

320 The first few terms are

$$P_1(\mathbf{S}) = \mathbf{S}, \quad (24)$$

$$P_2(\mathbf{S}) = \mathbf{S}^2 + \beta \mathbf{S}, \quad (25)$$

$$P_3(\mathbf{S}) = \mathbf{S}^3 + 2\beta \mathbf{S}^2, \quad (26)$$

$$P_4(\mathbf{S}) = \mathbf{S}^4 + 3\beta \mathbf{S}^3 + \beta^2 \mathbf{S}^2. \quad (27)$$

321 Therefore, residual reuse no longer applies only the single power $\mathbf{S}^\ell$; instead, it produces a mixture of
322 lower-order and higher-order propagation terms. In spectral coordinates, each eigen-component evolves
323 as

$$z_i^{(\ell+1)} = \lambda_i(z_i^{(\ell)} + \beta z_i^{(\ell-1)}), \quad (28)$$

324 which is fundamentally different from the plain recurrence $z_i^{(\ell)} = \lambda_i^\ell z_i^{(0)}$. The plain model attenuates every
325 non-dominant mode with a pure monomial factor, whereas the residual model preserves a polynomial
326 combination of powers of λ_i . As a result, informative mid-frequency components are not suppressed as
327 aggressively at the same depth.

328 A.3 Implication for Over-smoothing

329 The derivation above suggests the following interpretation. Plain stacking behaves like a repeated low-pass
330 filter whose order increases monotonically with depth. Residual reuse changes that filter into a polynomial
331 response that still smooths the graph, but preserves more low-order propagation information from earlier
332 layers. This does not eliminate smoothing, yet it slows the rate at which node representations collapse to
333 an almost indistinguishable subspace. In other words, residual reuse can improve depth tolerance because
334 it reduces the dominance of the highest propagation order.

335 A.4 Graph-Level Residual Passing

336 The same intuition extends to graph-level residual propagation. In the graph-conditioned variants, a
337 pooled summary from the previous block is broadcast back to the current block, so the hidden state is no
338 longer determined only by repeated applications of $\mathbf{S}$. A simplified affine form is

$$\mathbf{H}_t^{(\ell+1)} = \mathbf{S}\mathbf{H}_t^{(\ell)} + \mathbf{1}(\mathbf{g}_{t-1})^\top, \quad (29)$$

339 where $\mathbf{g}_{t-1}$ is the preceding graph summary. The additive term injects a lower-order global context at every
340 block and prevents the representation from being governed solely by deeper powers of the propagation
341 operator. This offers a second pathway for delaying over-smoothing, although the empirical sections
342 show that graph-level reuse is more sensitive to dataset structure and computational cost than the lighter
343 node-level residual design.

344 A.5 Scope of the Derivation

345 This appendix uses a linearized analysis and therefore abstracts away nonlinear activations, dropout,
346 optimizer dynamics, and operator-specific details. It should be read as an explanatory mechanism rather
347 than as a complete proof. Still, it aligns with the empirical observation that node-level residual reuse
348 remains more stable than plain stacking as depth increases, and it provides a compact theoretical rationale
349 for why reusing earlier states can alleviate over-smoothing.

B COMPLETE BENCHMARK TABLES

For completeness, we reproduce the final full-suite benchmark tables in the appendix. These are the same results discussed in Section , but collecting them here makes the full comparison easier to inspect during review.

Table 6. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 7. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Table 8. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

REFERENCES

- [1] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- [2] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.
- [3] Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.
- [4] Cătălina Cangea, Petar Veličković, Nikola Jovanović, Thomas Kipf, and Pietro Liò. Towards sparse hierarchical graph classifiers. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 165–176, 2018.
- [5] Kamalakkannan Charulekha, Agnishwar Girigoswami, Koyeli Girigoswami, Mani Manoj, Pragya Pallavi, Pemula Gowtham, Asirvatham Alwin Robert, and Arumugam Vijaya Anand. Uncovering complicated plant biological networks through the assistance of artificial intelligence tools. *AI in Plant Science and Precision Agriculture*, pages 25–54, 2025.
- [6] Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020.

- [7] Hongwei Du, Yiyuan Zhang, Peijie Zhang, Bin Gu, Xusheng Wei, Jiadong Guo, Ang Li, Zhenhua Wang, Tianyi Ma, et al. Densenn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules. *npj Computational Materials*, 10(1):304, 2024.
- [8] David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Advances in neural information processing systems*, pages 2224–2232, 2015.
- [9] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.
- [10] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- [11] Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.
- [12] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.
- [13] Vladimir Gligorić, P. Douglas Renfrew, Tomasz Kosciół, Julia Koehler Leman, Daniel Berenberg, Tommi Vatanen, Chris Chandler, Bryn C. Taylor, Ian M. Fisk, Hera Vlamakis, Ramnik J. Xavier, Rob Knight, Kyunghyun Cho, and Richard Bonneau. Structure-based protein function prediction using graph convolutional networks. *Nature Communications*, 12(1), 2021.
- [14] Marco Gori, Gabriele Monfardini, and Franco Scarselli. A new model for learning in graph domains. *Proceedings of IJCNN*, 2005.
- [15] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [17] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [18] Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.
- [19] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. Self-attention graph pooling. *arXiv preprint arXiv:1904.08082*, 2019.
- [20] Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.
- [21] Guohao Li, Elias Müller, Abbas Thabet, and Bernard Ghanem. Training graph neural networks with 1000 layers. In *International Conference on Machine Learning*, pages 6403–6413, 2021.
- [22] Shuyu Li, Muhan Zhang, Ming Jin, Han Zhang, Rex Ying, Zhenguang Li, and Junchi Yan. A non-asymptotic analysis of oversmoothing in graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2023.
- [23] Yucheng Li, Xiangtian Wang, Zhang Wang, and Jia Li. Learning graph normalization for graph neural networks. *Neurocomputing*, 486:140–151, 2022.
- [24] Zhipeng Li, Hao Chen, Zhiling Zeng, Jun He, Xianling Mao, Ning Lv, and Li Chen. Graph pooling for graph-level representation learning: a survey. *Artificial Intelligence Review*, 57(8):1–51, 2024.
- [25] Chaoyu Liu, Xixi Wang, Yushun Li, Yue Wang, and Jun Gao. Graph pooling for graph neural networks. In *International Joint Conference on Artificial Intelligence*, pages 4254–4261, 2023.
- [26] Christopher Morris, Roger Wattenhofer, and Kristian Kersting. TUDataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.
- [27] Tran Gia Bao Ngo, Christophe Liseron-Monfils, Shreyan Das, Jordan Ubbens, Paula Ashe, and David Konkin. Improving plant functional annotation from knowledge graphs using graph neural networks. *bioRxiv*, 2025.

- 424 [28] Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint*
425 *arXiv:2006.13604*, 2020.
- 426 [29] Manon Reau, Nicolas Renaud, Li C Xue, and Alexandre M. J. J. Bonvin. Deeprank-gnn: a graph
427 neural network framework to learn patterns in protein–protein interfaces. *Bioinformatics*, 39(1),
428 2022.
- 429 [30] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph
430 convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.
- 431 [31] Jianan Sui, Jiazi Chen, Yuehui Chen, Naoki Iwamori, and Jin Sun. Identification of plant vacuole
432 proteins by using graph neural network and contact maps. *BMC Bioinformatics*, 24(1):1–16, 2023.
- 433 [32] Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig,
434 and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint*
435 *arXiv:2111.14522*, 2021.
- 436 [33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua
437 Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*,
438 2018.
- 439 [34] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural
440 networks? In *International Conference on Learning Representations (ICLR)*, 2019.
- 441 [35] Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representa-
442 tion learning on graphs with jumping knowledge networks. In *International Conference on Machine*
443 *Learning*, pages 5449–5458, 2018.
- 444 [36] Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec.
445 Hierarchical graph representation learning with differentiable pooling. In *Advances in neural*
446 *information processing systems*, pages 4800–4810, 2018.
- 447 [37] Ronghui You, Shuwei Yao, Hiroshi Mamitsuka, and Shanfeng Zhu. Deepgraphgo: graph neural
448 network for large-scale, multispecies protein function prediction. *Bioinformatics*, 37(Suppl. 1):i262–
449 i271, 2021.
- 450 [38] Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint*
451 *arXiv:2009.10698*, 2020.